HearDesk

Technical Design Document

Application structure • Data storage • Access control

Website feedback. Ready for your team.

A personal full-stack engineering project

Prepared by Muhammad Danu Susyanto

Version 1.0 • 13 October 2026

Portfolio edition describing the implemented app, with test examples and dated results.

Contents

Contents 2
1. About the design 3
2. Terms used in the design 3
3. Application topology 4
4. Module responsibilities and dependency direction 4
5. Why these design choices were made 5
6. Company authorization and transactions 5
7. Core data relationships 6
8. Data dictionary: application records 8
9. Other supporting records 8
10. Submission and retry handling 8
11. Private tracking and the visitor response 9
12. Public API contract 10
13. Company API contract 11
14. Errors, files and delayed responses 11
15. Operation and design limits 11
16. Verification and source references 12

1. About the design

This document explains the HearDesk features and code reviewed for this portfolio. The results describe the recorded version and test environment. They do not represent a customer contract, independent certification or measured business results.

This Technical Design Document explains how the implemented application works. The reviewed source-code version is Git commit afb4f54a83eafcd0f10232adbbeeae92292c293a.

HearDesk runs as one application divided into feature modules. This is a modular monolith. Modules keep related code together, while the application can still be deployed as one unit. The modules are not separate network services.

Package versions in the reviewed code: Next.js 16.3.8, React 19.3.0, Prisma 7.10.0, Better Auth 1.7.7, pg 8.23.1, Zod 4.6.5, Sharp 0.35.5, Playwright 1.63.0, Vitest 5.0.3 and GSAP 3.15.0. The Node engine requirement is ^22.18.0 || >=24.11.0. This records project configuration rather than a support commitment from those libraries.

2. Terms used in the design

Table 1. Terms used in the design

Term	Meaning
Company / Site	A Company is a team workspace. A Site is one website registered in that workspace. Organization is the database model behind Company.
Origin	The protocol, hostname and port of a website, such as https://example.com. A path or query is not part of the origin.
iframe	An embedded page inside another page. The widget form is served by HearDesk inside the website.
API and DTO	An API is an interface used to send requests. A DTO (data transfer object) is the selected set of fields returned to a caller.
Use case / port / adapter	A use case performs a business action. A port defines an operation it needs. An adapter implements that operation using a database, file store or other service.
Repository	Code that reads or writes stored records and applies the required query restrictions.
Transaction / unit of work (UoW)	A transaction groups database changes so they succeed or fail together. The Company UoW supplies the access checks and repositories used inside that operation.
Hash / fingerprint	A hash is a value calculated from input. A fingerprint is a hash used here to compare submission content. Hashing is not reversible encryption.
Bearer code	A secret that grants access to its holder. Anyone with the tracking code can read the corresponding report.
Idempotency	Repeating the same identified request does not create another report or reply. The request identity and content must match.
Reservation / quota	A reservation temporarily claims capacity before a report is saved. A quota is a configured usage limit.
PK / FK / nullable	Primary key identifies a record. Foreign key links it to another record. Nullable means the field may be absent.

3. Application topology

The website owner registers a Site and adds the supplied script. The script opens the HearDesk form in an iframe. HearDesk then checks the input, stores the report and provides the team and visitor screens. The host website does not need its own feedback backend or review dashboard.

Setup requires an available HearDesk service, the correct Site key, a matching registered origin and host security settings that permit the script and iframe. Content Security Policy (CSP) is the browser policy that can restrict which scripts and embedded pages a website loads.

The browser calls Next.js routes. The routes validate requests and call application logic. Database adapters use Prisma and PostgreSQL, while private screenshot files use separate storage. Company membership and permissions control team access. The platform console has its own service-management checks.

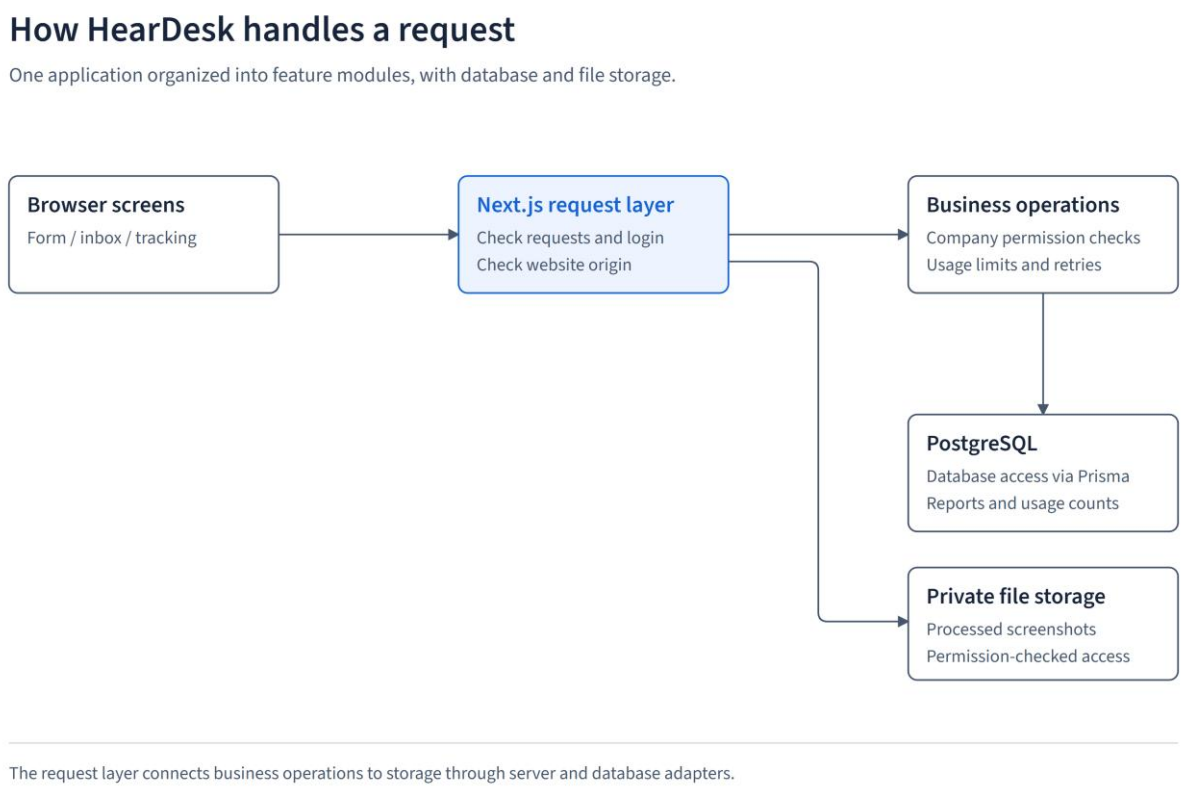


Figure 1. Application topology

4. Module responsibilities and dependency direction

Table 2. Module responsibilities and dependency direction

Area	Responsibility	Source
identity	Sign-in, sessions, email verification and the current user	src/modules/identity
companies	Company membership, roles, ownership and active status	src/modules/companies
websites	Website addresses, widget codes and Company ownership	src/modules/websites
feedback	Submission, report access, replies and report summaries	src/modules/feedback

Area	Responsibility	Source
usage	Report and storage limits, reservations and counts	src/modules/usage
platform	Service-management views, actions and audit records	src/modules/platform
HTTP / UI	Request checks, responses and user interaction	src/app, src/components, src/server
Composition	Connects business operations to access checks and storage adapters	src/server/composition

5. Why these design choices were made

Table 3. Why these design choices were made

Choice	Reason	Trade-off
Script and iframe	Reuse one feedback system across existing websites.	The website still needs the snippet, correct registration and compatible security settings.
Modular monolith	Keep related code together and deploy one application.	Code checks and tests enforce module boundaries. They are not network isolation.
Company ownership	Follow each report's Site to its Company and check current membership.	Every relevant query and write must apply the Company restriction.
Capacity reservations	Prevent simultaneous requests or retries from bypassing usage limits.	Failures need cleanup of temporary records and any staged files.
Hash-only tracking	Check a code without storing its original value.	Lost codes cannot be recovered. The browser must retain the code for an unchanged retry.
Separate reply records	Make replies visible to visitors an explicit action distinct from internal notes.	The team manages two separate message histories.
Private screenshot storage	Serve screenshots only after a permission check.	A usable backup must include both database records and image files.
Native dialog and GSAP	Use browser dialog behavior with controlled animation and reduced-motion support.	Code-analysis tools may not immediately recognize newer HTML attributes.

6. Company authorization and transactions

createCompanyServices connects the Company access UoW to its repositories. For an operation, it receives the user (actor), Company ID and required permission. It checks the current account, verified email, membership, role and Company status. A Company ID supplied by a request is not trusted as proof of access.

The FSD contains the Company permission matrix. Writing a public reply uses feedback:note, the permission held by OWNER, ADMIN and AGENT. Here, public means visible through the report's private tracking link.

Platform Owners can read their own Company's reports through their Company membership. Reading another Company's reports requires membership and permission in that Company. The server applies these checks regardless of platform role. Older /api/admin routes resolve the Company and call the same services.

Repository queries filter on Site.companyId. A missing report, an unauthorized report or a report without tracking access cannot receive a public reply. Supported platform-management actions record an audit event. This is not a claim that every report change has its own audit event.

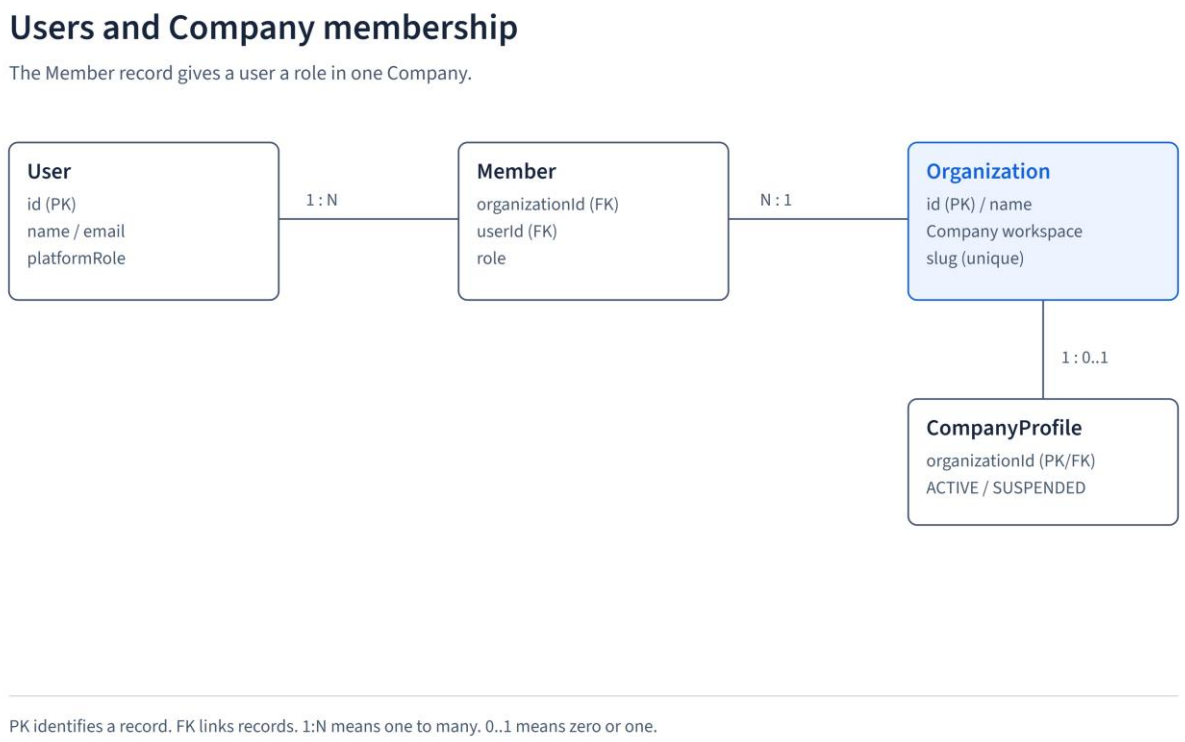


Figure 2. Company authorization and transactions

7. Core data relationships

The entity relationship diagram (ERD) shows records and how they connect. It is split into two diagrams so the labels remain readable. Organization stores the Company, Member links a user to a Company, and Site links reports to the correct Company.

Each Feedback record may have one Attachment and many InternalNote and PublicReply records. An internal note is team-only. A PublicReply is visible to someone with that report's private code. The following tables name the important database fields and constraints.

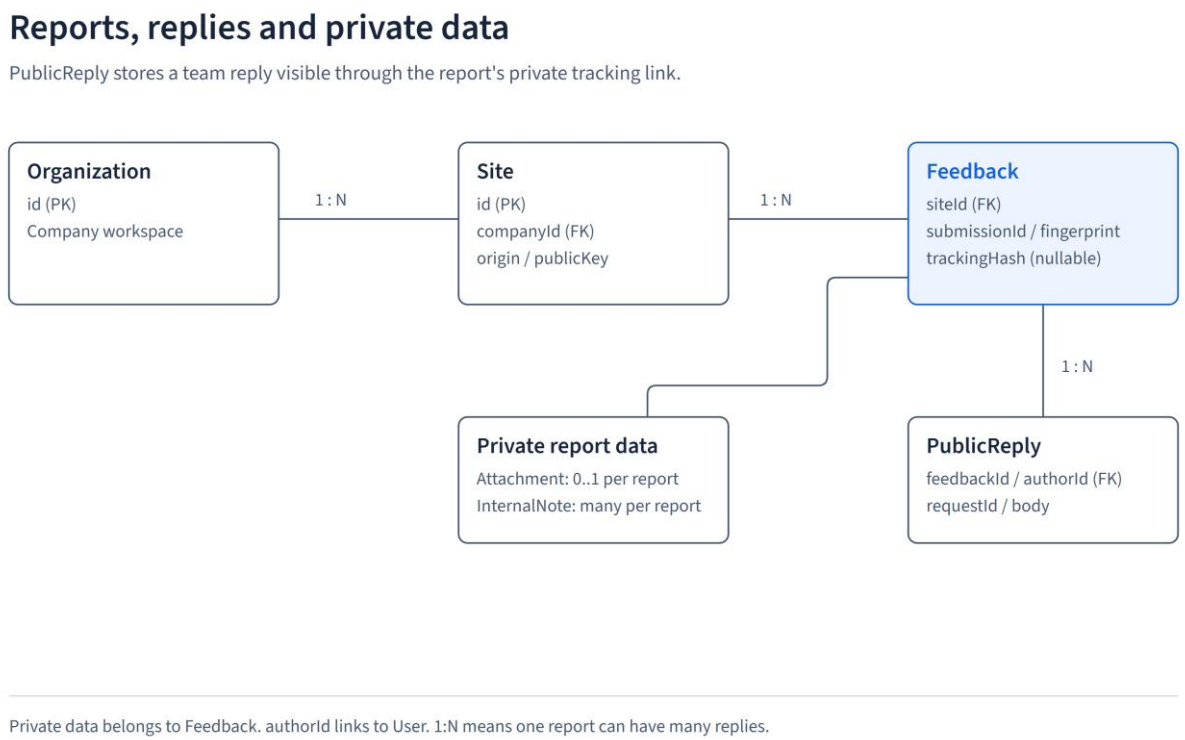


Figure 3. Core data relationships

8. Data dictionary: application records

Table 4. Data dictionary: application records

Model	Key fields / constraints	Purpose
Organization / Member	Organization.id unique Member(organizationId,userId)	Company workspace and user membership. The same user cannot have duplicate memberships in one Company.
CompanyProfile	organizationId PK/FK ACTIVE/SUSPENDED	Whether the Company is active or suspended.
Site	id, companyId, unique publicKey unique(companyId,origin)	Registered website and widget settings.
Feedback	id, siteId, fingerprint unique(siteId,submissionId) nullable unique trackingHash	Report content and status, with request matching and optional private tracking.
Attachment	unique feedbackId, unique storage key filename, mime, size	Private image-file details. Image bytes are stored separately.
InternalNote	feedbackId, authorId, body, createdAt	Messages for the team only.
PublicReply	feedbackId, authorId unique(feedbackId,requestId)	Replies visible through tracking. The writer's identity is stored privately.
CompanyLimits / Usage	Company FK, limits, storedBytes	Allowed usage and the number of stored bytes.
ReportUsagePeriod / Reservation	UTC period, expiry unique(siteId,submissionId)	Monthly report counts and temporary claims for in-progress submissions.
AuditEvent	actorSnapshot, action, resource optional Company, timestamp	Who performed a supported management action, what it affected and when.

9. Other supporting records

User, Session, Account and Verification support Better Auth sign-in and email verification. A session identifies an ongoing login. Session tokens and account secrets remain private. Invitation records support team invitations.

CompanyCreationRequest keeps the identity and content fingerprint needed to avoid duplicate Company creation on retry. RateBucket and rateLimit count requests for application and authentication limits. StorageBudget and UploadReservation track available storage and temporary uploads.

The full schema, including which fields are optional and which indexes help queries, is in prisma/schema.prisma. The diagrams omit some support tables for readability rather than implying those tables do not exist.

10. Submission and retry handling

Before sending a report, the widget creates a submissionId (a UUID used to identify the request) and a 24-byte random tracking code. Multipart means the request can carry form text and an image file together. readFeedbackForm checks the allowed fields, rejects repeated field values, requires multipart input and limits the complete request to 6 MiB. feedbackInput then checks the content.

The server calculates a SHA-256 fingerprint from the checked feedback fields, image hash and tracking-code hash when provided. SHA-256 produces a fixed-length value used for comparison. Reusing the same Site and

submissionId with matching content is idempotent, so it does not create another report. Changed content or a changed tracking code under the same identity returns a conflict instead of replacing the saved report.

Older clients may omit trackingCode and continue to submit without tracking access. A submission UUID alone cannot reveal or replace the visitor's tracking code.

submitVisitorFeedback finds the Site and Company, reserves capacity, prepares an image if present, and saves the report and usage totals. On failure it releases reservations and removes prepared files when needed. A new saved report returns HTTP 201. A matching retry returns HTTP 200 and the same caller-supplied tracking code. An empty screenshot is treated as no screenshot.

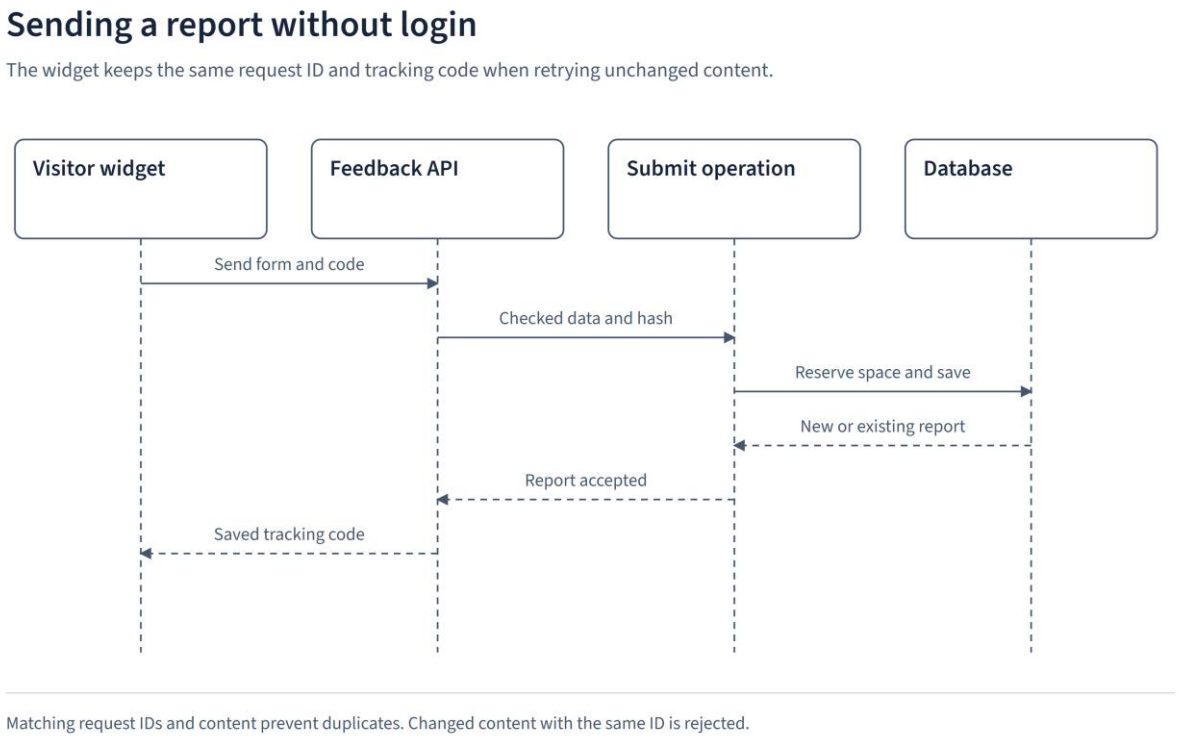


Figure 4. Submission and retry handling

11. Private tracking and the visitor response

The link is /track#code=<secret>. The part after '#' is a URL fragment, which the browser does not send as the page request path or query. The tracking page reads it and sends the code in the body of POST /api/feedback/track. It omits cookies and asks for no caching. This reduces exposure in URL logs, but anyone holding the link or code still has access to the report.

The server accepts only the expected JSON fields, checks the request origin, applies request limits, hashes the code and searches for the matching report in an active Company. The DTO, or response field set, contains website name, message, category, current status, timestamps and replies. Each reply includes its ID, text and timestamp, without the writer's identity.

The response excludes the report ID, Company ID, email, page URL, screenshot and internal notes. Cache-Control: private, no-store prevents storage by compliant caches. X-Content-Type-Options: nosniff tells browsers not to guess another response type. Page headers also control referrer information.

There is no email recovery for a lost code. Visitors must save their private tracking link or code, and documentation screenshots must not expose a live code.

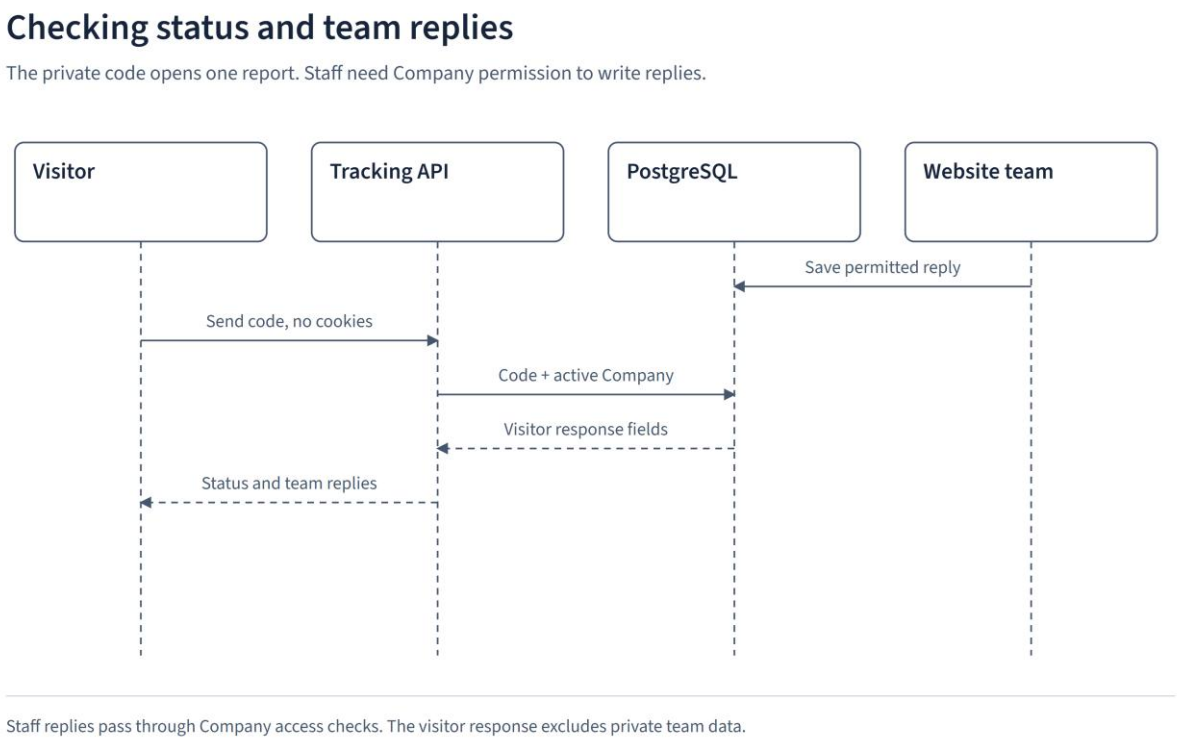


Figure 5. Private tracking and the visitor response

12. Public API contract

An API contract describes the request fields and the server's response. HTTP 2xx codes indicate success. The listed 4xx and 5xx codes identify rejected requests or server failures. Routes containing [key] or [companyId] use the actual value at that position.

Table 5. Public API contract

Method / route	Input	Response / important failures
POST /api/feedback	Multipart: siteKey, submissionId, type, message. Optional: email, pageUrl, screenshot, trackingCode.	201 new report or 200 matching retry. 400 invalid input, 404 missing/inactive widget at first lookup, 413 too large, 415 unsupported media, 409 changed/in-progress/invalid reservation, 422 Company allowance exhausted, 403 suspension during processing, 429 too many requests, 503 capacity/processing/unexpected failure.
POST /api/feedback/track	JSON with one code of 48 lowercase hexadecimal characters.	200 selected visitor fields. 400 invalid request, 403 origin rejected, 404 unknown code or inactive Company, 429 too many requests.
GET /widget/[key]	Site public key and optional parentOrigin.	The configured form or a not-found response when the key/origin is not allowed.
GET /api/health	No customer data.	Readiness status without report content.

13. Company API contract

These routes require a signed-in user and Company permissions. The prefix for shortened routes is /api/companies/[companyId]. Each returned report must belong to that Company.

Table 6. Company API contract

Method / route	Permission / request	Behavior
GET /api/companies/[companyId]/feedback	feedback:read, checked filters and pagination	Reports for the authorized Company, divided into pages.
GET/PATCH .../feedback/[feedbackId]	feedback:read or feedback:status, supported status value	Read detail or save status. Reject reports outside the allowed Company.
POST .../feedback/[feedbackId]/notes	feedback:note, body of 1 to 5,000 characters	201 team-only note.
POST .../feedback/[feedbackId]/replies	feedback:note, body and requestId UUID	201 team reply. Matching retries return the same record, while changed text under the same requestId is rejected.
GET .../attachments/[attachmentId]	feedback:read	Private image download after permission checks.
GET .../feedback/statistics	Company feedback-read permission	Checked monthly report totals.
GET .../feedback/export	Company feedback-read permission	Excel report for a supported reporting period.
/api/admin/*	Older routes with explicit Company context	Call the same restricted services rather than providing access to every Company's inbox.

14. Errors, files and delayed responses

Zod checks input and returns field errors when it is invalid. HttpError carries an intended status and message. Company errors distinguish missing login, insufficient permission and a resource that cannot be found. Unexpected errors return a generic message and request ID rather than database details.

The error response's error.requestId is a server-generated reference for troubleshooting that failure. It is separate from the client-generated report submissionId and reply requestId. Keep the original client-generated ID when retrying unchanged content. Do not use the error reference as a retry ID.

Sharp processes screenshot images into the supported stored format. Image files are private, with authorized downloads through the server. Temporary uploads and reservations are cleaned up after failures. Database records alone are not a complete backup because the image bytes live in separate files.

The tracking component numbers its requests. A delayed result from an earlier code lookup is ignored after a newer lookup starts. The report-detail page similarly avoids displaying a previous report under a new report ID. Reply retries retain their request ID until the text changes or the send succeeds.

GSAP animations remove their effects when components are no longer active and respect reduced-motion preferences. The UI displays a changed report status only after the server saves it.

15. Operation and design limits

Use the project's Node version requirement and generate the Prisma client before building. Database migrations are versioned schema changes. Review them and back up both the database and screenshot files before applying them. The application's normal database account and the account allowed to migrate the schema have separate privileges.

The E2E runner checks that it targets ureport_test before clearing test data. It must never point to the application database. Local email is written to a test mailbox rather than delivered to a real recipient.

Visitors cannot reply through tracking, lost codes cannot be recovered, and status history is not implemented. These are current design limits. Hosting details and a public application address are not claimed by this documentation.

16. Verification and source references

Unit tests check small business operations. Component tests exercise individual UI components. Integration tests use PostgreSQL to check real queries, permissions and usage counts. End-to-end tests drive the browser against the running application. Each type checks a different layer.

SonarQube imports LCOV files, a format that records which code tests executed. These figures are separate from browser scenario pass counts. The companion test and quality reports retain dates and results.

Sources: prisma/schema.prisma, src/server/composition/company-services.ts, src/modules/feedback/application/submit-visitor-feedback.ts, src/modules/usage/infrastructure/prisma-visitor-usage.ts, src/proxy.ts, public/widget.js, docs/adr/0001-modular-monolith.md and docs/adr/0002-company-data-ownership.md.